AI-NATIVE GROCERY MERCHANT

Document 06 — Policy & Mandate Specification

Development-only specification • Locked MVP • Deterministic authorization, mandate enforcement and failure handling

> **Core authority principle: The AI is intelligent but untrusted. The policy engine is deterministic, server-side and authoritative for every money action.**

Document status: LOCKED DEVELOPMENT SPECIFICATION

Authorization chain: AI proposal → server-side revalidation → policy engine → ALLOW/DENY → Razorpay execution → server verification → audit.

1. Purpose and Scope

This document defines the mandate and policy system that controls whether a proposed grocery purchase is authorized. It specifies mandate structure, policy rules, evaluation order, denial semantics, revalidation requirements, security boundaries, audit requirements and recovery behavior.

The policy engine is intentionally deterministic. It does not use LLM reasoning to decide whether money movement is authorized.

1.1 In scope

• Mandate creation and representation.

• Mandate validity and constraint enforcement.

• Category and per-item restrictions.

• Stock and authoritative product-state checks.

• Incentive validity checks relevant to final pricing.

• Authoritative final amount calculation.

• Maximum-spend enforcement.

• ALLOW/DENY policy decisions.

• Structured denial reasons.

• Pre-payment and immediate pre-checkout revalidation.

• Recovery handoff to the optimization agent.

• Policy decision audit trail.

1.2 Out of scope

• LLM-based authorization.

• Changing a user's mandate automatically.

• Autonomous refunds, payouts or banking actions.

• Production-money authorization.

• Multi-merchant authorization.

• Full ACP/AP2/UAP certification or implementation.

2. Security and Trust Model

The policy engine is the financial authorization boundary between the intelligent planning system and the payment system.

```
AI / LLM
= UNTRUSTED PROPOSER

Frontend
= UNTRUSTED INPUT SURFACE

Backend commerce services
= AUTHORITATIVE STATE

Policy Engine
= AUTHORIZATION AUTHORITY

Razorpay integration
= PAYMENT EXECUTION BOUNDARY
```

2.1 Non-negotiable rule

No money action may occur from an AI decision alone. A proposal must pass deterministic server-side validation and policy authorization before the backend creates the Razorpay order.

2.2 Policy engine must not trust

• Client-provided final amounts.

• LLM-generated prices.

• LLM-generated stock claims.

• LLM-generated voucher validity.

• Frontend claims of mandate authorization.

• Frontend claims of payment success.

3. Mandate Model

A mandate represents the authority granted to the shopping agent for a defined purpose and period.

```
{
"agent_id": "agent-001",
"mandate_id": "mandate-001",
"max_spend": 1000,
"currency": "INR",
"allowed_categories": ["grocery"],
"max_per_item": 300,
"valid_until": "2026-09-07T23:59:59Z",
"purpose": "pasta ingredients for 4",
"status": "ACTIVE"
}
```

3.1 Mandate fields

Field	Rule
agent_id	Identifies the authorized agent/session.
mandate_id	Stable identifier for authorization checks and audit.
max_spend	Hard upper bound on final payable amount.
currency	INR for the locked MVP.
allowed_categories	Hard category allow-list.
max_per_item	Maximum authorized amount for an individual item/line according to the defined backend interpretation.
valid_until	Mandate expiration timestamp.
purpose	Human-readable reason/context for the mandate.
status	ACTIVE/INACTIVE or equivalent backend state.

4. Mandate Lifecycle

4.1 Lifecycle states

```
CREATED
↓
ACTIVE
↓
■■■ EXPIRED
■■■ REVOKED
■■■ CONSUMED / CLOSED (if application rules require closure)
```

4.2 Lifecycle rules

• Only an authorized server-side operation may create, update or revoke a mandate.

• The AI cannot increase max_spend.

• The AI cannot expand allowed categories.

• The AI cannot extend valid_until.

• The AI cannot change max_per_item.

• Expired or revoked mandates cannot authorize new payment actions.

• Mandate state must be re-read or validated immediately before authorization.

For the MVP, mandate changes are a user/system responsibility and are never an agent recovery mechanism.

5. Policy Evaluation Contract

The policy engine accepts a proposed basket reference plus authoritative commerce state and returns a deterministic result.

```
{
"mandate_id": "mandate-001",
"basket_id": "basket-quality-001",
"category": "grocery",
"items": [
{
"sku_id": "sku-001",
"quantity": 2,
"line_amount": 4200
}
],
"final_payable": 87400,
"currency": "INR"
}
```

All monetary values should be represented internally in minor currency units, such as paise. ■874 is represented as 87400.

5.1 Policy result

```
{
"decision": "ALLOW",
"reason_code": "AUTHORIZED",
"mandate_id": "mandate-001",
"basket_id": "basket-quality-001",
"final_payable": 87400,
"policy_version": "policy-v1",
"evaluated_at": "2026-09-02T12:00:00Z"
}
```

6. Mandatory Evaluation Order

Policy checks must execute in a deterministic order. Earlier failures should prevent later money-action steps.

Order	Check	Failure code
1	MANDATE_VALID	MANDATE_INVALID
2	CATEGORY_ALLOWED	CATEGORY_NOT_ALLOWED
3	STOCK_AVAILABLE	STOCK_UNAVAILABLE
4	MAX_PER_ITEM	MAX_PER_ITEM_EXCEEDED
5	INCENTIVE/VOUCHER VALIDITY	INCENTIVE_INVALID
6	FINAL_AMOUNT_CALCULATION	AMOUNT_CALCULATION_FAILED
7	MAX_SPEND	MAX_SPEND_EXCEEDED
8	Decision	ALLOW / DENY
9	Audit	Record decision
10	Execution gate	Proceed only after ALLOW

6.1 Ordering rationale

The sequence ensures that authorization is based on current mandate, category, stock, item-level constraints, valid incentives and an authoritative final payable amount. Maximum-spend validation must use the final payable, not the gross subtotal.

7. Rule 1 — Mandate Validity

The policy engine must verify that the mandate exists, belongs to the relevant agent/user context, is active and has not expired.

7.1 Conditions

• mandate_id exists.

• Mandate belongs to the current authorized context.

• status is ACTIVE.

• Current time is before valid_until.

• Mandate has a valid currency and required constraints.

```
if mandate_missing:
DENY(MANDATE_INVALID)

if mandate.status != ACTIVE:
DENY(MANDATE_INVALID)

if now >= mandate.valid_until:
DENY(MANDATE_INVALID)
```

8. Rule 2 — Category Authorization

Every purchased item must belong to a category allowed by the mandate.

8.1 Rules

• Use authoritative catalog category data.

• Do not trust an LLM-supplied category label.

• If any item is outside the allowed category, deny or remove it through a valid re-optimization flow.

• Do not broaden the mandate to accommodate a product.

```
allowed = set(mandate.allowed_categories)

for item in basket.items:
if catalog[item.sku_id].category not in allowed:
DENY(CATEGORY_NOT_ALLOWED)
```

9. Rule 3 — Stock Availability

The policy engine must verify current stock for every item at the authorization boundary.

9.1 Rules

• Use backend/catalog inventory state.

• Requested quantity must be available.

• Stale planning results must not authorize unavailable products.

• If stock changed, return a structured denial so the optimizer can replace the affected item.

Stock checks should occur again immediately before creating the payment order when the implementation permits a race between policy evaluation and checkout.

10. Rule 4 — Maximum Per Item

The mandate may define a maximum authorized amount per item. This prevents a valid overall basket from containing a single disproportionately expensive line.

10.1 Interpretation

The implementation must define whether max_per_item applies to unit price or line amount and use that definition consistently across catalog, cart and policy services. For the locked MVP, the preferred interpretation is the authoritative amount attributable to the item line being authorized.

```
if item.line_amount > mandate.max_per_item:
DENY(MAX_PER_ITEM_EXCEEDED)
```

The exact representation must remain consistent with the minor-unit money convention.

11. Rule 5 — Incentive and Voucher Validity

An incentive may only reduce the final payable if the backend confirms that it is valid and applicable to the selected basket.

11.1 Required checks

• Voucher/reward exists.

• It is active and unexpired.

• User/session is eligible.

• Basket meets minimum-spend and category requirements.

• Product exclusions are satisfied.

• Redemption limits are satisfied.

• Discount amount is calculated by the authoritative incentive service.

• The same incentive is not applied twice.

The policy engine validates incentive correctness. The AI decides whether using a valid incentive is valuable; it cannot make an invalid incentive valid.

12. Rule 6 — Final Amount Calculation

The final payable must be calculated by the backend from authoritative item prices, quantities and valid incentives.

```
gross_amount
= authoritative sum(line prices × quantities)

discount_amount
= authoritative applicable incentives/rewards

final_payable
= gross_amount - discount_amount

final_payable >= 0
```

12.1 Prohibited behavior

• Do not accept final_payable from the browser as authoritative.

• Do not accept model arithmetic as authoritative.

• Do not subtract a discount twice.

• Do not use stale quote data after a material basket change.

13. Rule 7 — Maximum Spend

Maximum spend is a hard ceiling. The policy engine compares the authoritative final payable against the mandate's max_spend.

```
if final_payable > mandate.max_spend:
DENY(MAX_SPEND_EXCEEDED)
else:
continue
```

13.1 Critical rule

The budget is a ceiling, not a spending target. A ■650 basket under a ■1000 mandate is valid. The system must not add products merely to approach ■1000.

13.2 Gross vs final payable

Max-spend authorization must use final payable after valid discounts/rewards, while gross amount and discount amount remain separately recorded for explainability and audit.

14. ALLOW Decision

ALLOW is returned only when every mandatory check succeeds.

```
ALLOW requires:
mandate_valid
AND category_allowed
AND stock_available
AND max_per_item_ok
AND incentives_valid
AND amount_calculated
AND final_payable <= max_spend
```

14.1 ALLOW is scoped

An ALLOW result authorizes the specific validated basket under the specific mandate and policy version. It is not a general authorization for arbitrary future purchases.

14.2 Execution gate

Only the backend payment boundary may consume an ALLOW result to create a Razorpay Test Mode order. The frontend and AI must not be treated as authorization authorities.

15. DENY Decision

DENY must be explicit, machine-readable and actionable.

```
{
"decision": "DENY",
"reason_code": "MAX_SPEND_EXCEEDED",
"message": "Final payable exceeds the authorized maximum spend.",
"recoverable": true,
"policy_version": "policy-v1"
}
```

15.1 Reason codes

Code	Meaning	Typical recovery
MANDATE_INVALID	Mandate missing, inactive, expired or unauthorized	Stop; user/system must resolve mandate
CATEGORY_NOT_ALLOWED	Item outside allowed category	Remove/replace item
STOCK_UNAVAILABLE	Required item/quantity unavailable	Refresh and replace/re-optimize
MAX_PER_ITEM_EXCEEDED	Item violates per-item limit	Replace/adjust pack
INCENTIVE_INVALID	Voucher/reward cannot be applied	Remove incentive and recalculate
AMOUNT_CALCULATION_FAILED	Authoritative quote unavailable/invalid	Retry quote safely; stop if unresolved
MAX_SPEND_EXCEEDED	Final payable exceeds mandate	Re-optimize within unchanged mandate

16. Recovery Contract

Policy denial is not an LLM failure. It is an expected control boundary. When the denial is recoverable, the agent receives the structured reason and can request a new optimization proposal.

16.1 Recovery flow

```
PROPOSAL
↓
```

```
POLICY
↓
DENY
↓
STRUCTURED REASON
↓
AGENT / OPTIMIZER
↓
NEW PROPOSAL
↓
AUTHORITATIVE QUOTE
↓
POLICY AGAIN
```

16.2 Example

Mandate max_spend = ■800. Selected basket final_payable = ■850. Policy returns DENY / MAX_SPEND_EXCEEDED. The agent must not increase the mandate. It may replace a product, improve pack efficiency or reconsider incentive choices, then produce a basket such as ■774 and request policy evaluation again.

17. Revalidation Before Payment

A previously allowed proposal may become stale. Therefore authorization must be tied to current state.

17.1 Required sequence

• User selects a basket.

• Backend loads authoritative basket/catalog state.

• Backend calculates a fresh quote.

• Policy engine evaluates the fresh quote.

• Only ALLOW permits payment-order creation.

• Payment order amount is taken from the authorized quote, not the client.

17.2 Race handling

If price, stock, incentive or mandate state changes after policy evaluation but before order creation, the backend must fail closed, re-quote and re-run policy rather than creating an order from stale authorization.

18. Policy Versioning

Every decision should record the policy version used for evaluation.

```
{
"policy_version": "policy-v1",
"ruleset_hash": "optional-integrity-hash",
"evaluated_at": "2026-09-02T12:00:00Z"
}
```

18.1 Rules

• Policy changes must be versioned.

• Historical decisions retain their original policy version.

• A changed policy must not silently reinterpret an old audit record.

• Deployment should include tests for every policy rule and reason code.

19. Idempotency and Duplicate Requests

Authorization endpoints must tolerate retries without producing inconsistent financial actions.

19.1 Requirements

• Use a stable request/idempotency key for repeated authorization attempts.

• Repeated evaluation of the same unchanged request should return a consistent result.

• Do not create multiple payment orders because a policy request was retried.

• Persist policy decision identifiers for audit correlation.

• Payment idempotency remains the responsibility of the Razorpay integration boundary as specified in Document 09.

20. Audit Trail

The policy engine must produce an immutable or append-only decision record sufficient to explain why a money action was allowed or denied.

20.1 Minimum audit fields

```
{
"event_type": "POLICY_DECISION",
"policy_decision_id": "pd-001",
"mandate_id": "mandate-001",
"basket_id": "basket-quality-001",
"decision": "DENY",
"reason_code": "MAX_SPEND_EXCEEDED",
"gross_amount": 92000,
"discount_amount": 4600,
"final_payable": 87400,
"max_spend": 80000,
"policy_version": "policy-v1",
"evaluated_at": "2026-09-02T12:00:00Z"
}
```

Audit records should preserve decision facts, identifiers and state references. They should not be editable by the AI.

21. Security Controls

• Policy evaluation occurs server-side.

• Mandates are retrieved from trusted storage.

• Catalog prices and stock are retrieved from trusted backend services.

• Money values are represented using integer minor units.

• Client-provided totals are ignored for authorization.

• AI-generated policy decisions are ignored.

• Policy service has no authority to accept arbitrary model instructions.

• Payment credentials/secrets are never exposed to the agent.

• All sensitive endpoints require authenticated server-to-server context.

• Authorization logs must not contain unnecessary secrets or payment credentials.

22. Frontend Boundary

The frontend may display policy state but cannot create authorization.

Frontend may	Frontend may not
Display mandate constraints	Modify mandate constraints
Display basket totals	Declare totals authoritative
Submit basket selection	Authorize payment
Display ALLOW/DENY	Override DENY
Start checkout after backend approval	Create a payment order directly using untrusted amount

23. Agent Boundary

The AI agent can request policy evaluation and react to the result, but it cannot influence the rule outcome.

Agent may	Agent may not
Request policy evaluation	Change policy rules
Explain denial	Override denial
Re-optimize basket	Change mandate
Choose alternative products	Change authoritative prices
Suggest incentive decisions	Invent incentive eligibility

24. Reference End-to-End Authorization

```
1. Agent generates Best Value + Best Quality.
2. User chooses one basket.
3. Backend loads selected basket.
4. Backend refreshes price/stock/incentive state.
5. Backend calculates authoritative gross/discount/final payable.
6. Policy validates:
a. mandate
b. category
c. stock
d. max per item
e. incentives
f. final amount
g. max spend
7. Policy records decision.
8. If DENY:
return reason to optimizer; no payment order.
9. If ALLOW:
create Razorpay Test Mode order from authorized amount.
10. Complete checkout.
11. Verify payment server-side / process webhook.
12. Record final transaction state in audit.
```

Financial invariant: There must be no path from LLM output directly to Razorpay order creation.

25. Testing Matrix

Test	Expected
Active mandate + valid basket under limit	ALLOW
Expired mandate	DENY / MANDATE_INVALID
Revoked mandate	DENY / MANDATE_INVALID
Unauthorized mandate context	DENY / MANDATE_INVALID
Disallowed category	DENY / CATEGORY_NOT_ALLOWED
Insufficient stock	DENY / STOCK_UNAVAILABLE
Per-item limit exceeded	DENY / MAX_PER_ITEM_EXCEEDED
Expired voucher	DENY / INCENTIVE_INVALID
Invalid voucher eligibility	DENY / INCENTIVE_INVALID
Quote calculation unavailable	DENY / AMOUNT_CALCULATION_FAILED
Final payable exactly equals max_spend	ALLOW
Final payable exceeds max_spend by ■1	DENY / MAX_SPEND_EXCEEDED
Gross > max_spend but valid discount makes final payable <= max_spend	Evaluate using final payable; ALLOW if all other rules pass
Client sends manipulated final amount	Ignore client amount; use authoritative quote
Policy denial followed by lower feasible basket	Second policy check can ALLOW
State changes after ALLOW but before payment order	Fail closed; revalidate

Test	Expected
Duplicate authorization request	Idempotent/consistent handling

26. Definition of Done

• Mandate structure is implemented and server-authoritative.

• Mandate validity is enforced.

• Allowed categories are enforced.

• Current stock is checked.

• Maximum per-item constraint is enforced.

• Voucher/incentive validity is checked deterministically.

• Final payable is calculated authoritatively.

• Maximum spend uses final payable.

• ALLOW/DENY results are structured and versioned.

• Every denial has a stable reason code.

• Recoverable denials can be passed back to the optimizer.

• Mandate cannot be modified by the AI.

• Policy cannot be overridden by frontend or AI.

• Authorization is revalidated immediately before payment.

• Client totals cannot influence authorization.

• Duplicate requests are handled safely.

• Policy decisions are auditable.

• No payment order can be created without policy ALLOW.

• All policy tests in the matrix pass.

• No locked MVP scope has been added.

27. Related Development Documents

Document	Relationship
01 — Development Master Spec	Locked product and engineering baseline
02 — Functional Requirements	Feature-level behavior
03 — System Architecture	Trust boundaries and component design
04 — AI Agent Specification	Agent behavior and tool boundaries
05 — Optimization Decision Specification	Basket and incentive optimization before authorization
07 — Data Model Specification	Mandate, policy, basket and audit persistence
08 — API Contract Specification	Policy and mandate endpoints
09 — Razorpay Integration Specification	Payment execution after policy ALLOW
10 — Testing & Acceptance Specification	End-to-end authorization and payment tests